



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 01

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

E – Environment: The environment in which the agent operates.
A - Actuators: The means the agent has to act.
S – Sensors: The way in which the agent senses the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.food_positions – Stores the locations of food pellets.
 self.opponents – Stores the positions of opponent agents.
 self.agent_pos – Stores the current position of the agent.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This environment is multi-agent as there is the main agent and multiple opponents in `self.opponents`. These enemies move in the grid individually and they can collide with the main agent affecting its performance. So it's a multi-agent environment, not

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all the percepts the agent has ever received . The agent uses this accumulated information to make a decision on its next action.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method is the "S" in the PEAS model (S=Sensors). It collects information from the environment (agent position, food detection, wall detection, opponent positions, score, remaining food) and provides the agent with these

6. (Evaluate) Based on the exact dictionary returned by `get_percept()` , is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

positions, food at current location, wall detection, score, remaining food etc. but does not give the complete state of the environment. The agent cannot see all food pellets or all walls unless it comes upon them. This means that the agent has partial

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance Measure (P) component is being actively updated.

Explanation: When the agent hits a wall, the statement `self.score -= 5` decreases the agent's score. The score is the performance measure used to evaluate how well the

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because a performance measure should penalize the agent for crashing into a wall. The penalty reflects how good or bad the agent is at navigating the maze. If you penalize it based on how many calculations it makes internally, it

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The problem becomes Partially Observe because the agent cannot see where the toxic traps are. The agent cannot know what it does not know, and that is why it makes the task harder.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding the `smells_toxin` sensor would give the agent extra information about the environment. Using this information, the agent can avoid getting close to the traps full of poison and therefore can avoid paying a penalty. By increasing the amount of

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

time the robot successfully picked up dirt, even if it was then dumped on the floor again. By strategically cleaning the dirt up and dumping it back on the floor, the robot could continue to achieve an infinite number of rewards. By this example, it

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!



FACULTY OF COMPUTING

